

Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Solutions: Exercise 5 - EZ-Diffusion Model (Color Judgement tasks)

In this exercise, we will have a look at estimating the parameters from the diffusion decision model using the hierarchical Bayesian implementation of the ez-Diffusion Model. This implementation allows for a fast parameter estimation while still providing acceptable recovery of three parameters of interest:

1. drift rate = the rate of evidence accumulation
2. boundary separation = the separation of decision thresholds
3. non-decision time = the time for non-decisional processes.

Your task is to load one of the data sets we prepared for this exercise, either `exercercise5_visual_search_task.txt` or `exercercise5_color_judgement_task.txt`.

The *Color Judgement* data set contains data of 50 participants that completed a color judgement task. In this task participants saw an array of blue and red dots and should decide if the array contained more blue or red dots. There were two difficulty levels, the easy level in which 60% of the dots were either blue or red, and the difficult level in which 52% of the dots were either blue or red.

The *Visual Search* data set contains data for 33 participants that completed the visual search task. In this task, participants were presented a color cue, a positive (the target color), a negative (the non target color) or neutral (not part of the visual search display). Participants had to look for a “C” that either looked upwards or downwards and report it’s direction with the arrow key.

```
# empty environment
rm(list = ls())

# Load required packages
library(bmm)
library(tidyverse)
library(here)
library(bayestestR)
```

1. Prepare data

First, we need to aggregate the data so that we can analyze our data with the `ezdm` using the `bmm` function. For this, we need to calculate the mean reaction time, the variance of the reaction time, the number of correct responses, and the number of trials in each of the four cells of the experimental design: easy - blue correct, easy - red correct, difficult - blue correct, and difficult - red correct.

```
ezdm_data <- read.table(here("data", "exercise5_color_judgement_task.txt"),
                        header = T, sep = ",")

ezdm_data_agg <- ezdm_data %>%
  filter(rt > 150, rt < 5000) %>%
  summarise(
    mrt = mean(rt/1000),
    vrt = var(rt/1000),
```

```

    n_correct = sum(correct),
    n_trials = n(),
    .by = c("participant_id", "condition", "correct_response")
  )

ezdm_data_agg$condition <- as.factor(ezdm_data_agg$condition)
contrasts(ezdm_data_agg$condition) <- contr.equalprior

```

2. Specify model object

Next, we need to specify the model object to link which data columns are used to identify the `ezdm`.

```

ezdm_model <- bmm::ezdm(mean_rt = "mrt",
                        var_rt = "vrt",
                        n_upper = "n_correct",
                        n_trials = "n_trials")

```

3. Specify the BMM formula

Now, we can consider which parameters are affected by the experimental manipulations. First, let's see how the `ezdm` parameters vary as a function of the difficulty manipulation.

```

ezdm_formula <- bmf(
  drift ~ 1 + condition + (1 + condition | participant_id),
  bound ~ 1 + (1 | participant_id),
  ndt ~ 1 + condition + (1 + condition | participant_id)
)

```

4. Estimate ezdm with the bmm function

Finally, we can estimate the model using our aggregated data, the `bmmmodel` object we created and the `bmmformula` specifying how the model parameters should change over experimental condition.

```

ezdm_fit <- bmm(
  formula = ezdm_formula,
  data = ezdm_data_agg,
  model = ezdm_model,
  cores = 4,
  sample_prior = TRUE
  #file = here("models", "ezdm_colortask")
)

```

```

## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 2 Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 3 Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 4 Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%] (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%] (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%] (Warmup)
## Chain 4 Iteration:  100 / 2000 [  5%] (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%] (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%] (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%] (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%] (Warmup)

```

```

## Chain 1 Iteration: 300 / 2000 [ 15%] (Warmup)
## Chain 4 Iteration: 300 / 2000 [ 15%] (Warmup)
## Chain 2 Iteration: 300 / 2000 [ 15%] (Warmup)
## Chain 1 Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 3 Iteration: 300 / 2000 [ 15%] (Warmup)
## Chain 4 Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 1 Iteration: 500 / 2000 [ 25%] (Warmup)
## Chain 2 Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 3 Iteration: 400 / 2000 [ 20%] (Warmup)
## Chain 4 Iteration: 500 / 2000 [ 25%] (Warmup)
## Chain 1 Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 2 Iteration: 500 / 2000 [ 25%] (Warmup)
## Chain 3 Iteration: 500 / 2000 [ 25%] (Warmup)
## Chain 4 Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 1 Iteration: 700 / 2000 [ 35%] (Warmup)
## Chain 2 Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4 Iteration: 700 / 2000 [ 35%] (Warmup)
## Chain 3 Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 1 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 2 Iteration: 700 / 2000 [ 35%] (Warmup)
## Chain 4 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 3 Iteration: 700 / 2000 [ 35%] (Warmup)
## Chain 1 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 2 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 3 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 1 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 3 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 2 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%] (Sampling)

```

```

## Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 1 finished in 22.4 seconds.
## Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4 finished in 23.2 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2 finished in 23.4 seconds.
## Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3 finished in 24.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 23.3 seconds.
## Total execution time: 24.2 seconds.

```

We have a look at the model output:

```
summary(ezdm_fit)
```

```

## Model: bmm::ezdm(mean_rt = "mrt",
##                   var_rt = "vrt",
##                   n_upper = "n_correct",
##                   n_trials = "n_trials")
## Links: drift = identity; bound = log; ndt = log; s = log
## Formula: drift ~ 1 + condition + (1 + condition | participant_id)
##          bound ~ 1 + (1 | participant_id)
##          ndt ~ 1 + condition + (1 + condition | participant_id)
##          s = 0
## Data: ezdm_data_agg (Number of observations: 200)
## Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##        total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~participant_id (Number of levels: 50)
##
##          Estimate Est.Error 1-95% CI u-95% CI Rhat
## sd(drift_Intercept)      0.49    0.05    0.40    0.61 1.00
## sd(drift_condition1)     0.43    0.05    0.35    0.54 1.00
## sd(bound_Intercept)      0.27    0.03    0.22    0.34 1.01
## sd(ndt_Intercept)        0.27    0.04    0.20    0.37 1.00

```

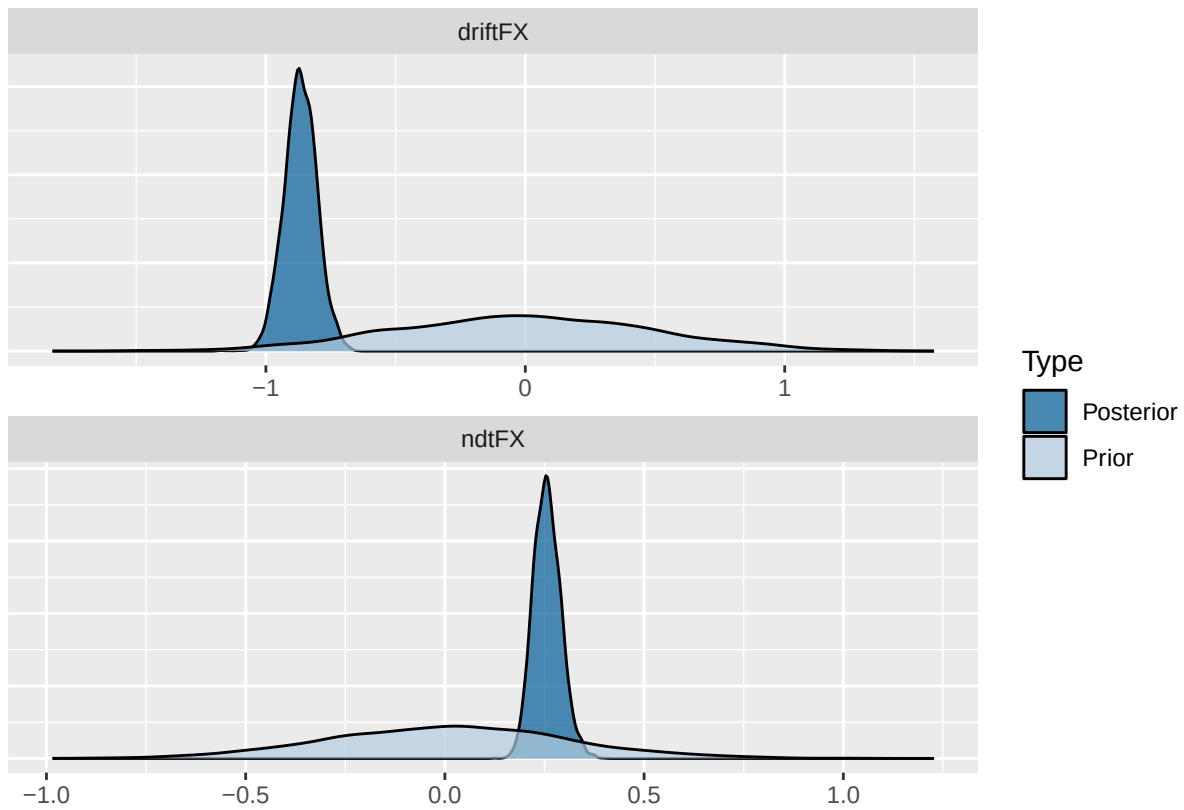
```
## sd(ndt_condition1)          0.16      0.04      0.09      0.23 1.00
## cor(drift_Intercept,drift_condition1) -0.93      0.03     -0.97     -0.85 1.00
## cor(ndt_Intercept,ndt_condition1)    -0.20      0.25     -0.65      0.32 1.01
##                               Bulk_ESS Tail_ESS
## sd(drift_Intercept)              767     1281
## sd(drift_condition1)             925     1368
## sd(bound_Intercept)              790     1147
## sd(ndt_Intercept)               1435     2227
## sd(ndt_condition1)              1084     1353
## cor(drift_Intercept,drift_condition1) 1352     2295
## cor(ndt_Intercept,ndt_condition1)    1664     2236
##
## Regression Coefficients:
##           Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## drift_Intercept      1.21      0.07    1.07    1.35 1.01      314      702
## bound_Intercept      0.63      0.04    0.55    0.71 1.01      303      524
## ndt_Intercept       -0.85      0.04   -0.94   -0.77 1.00      827     1524
## drift_condition1    -0.87      0.06   -0.99   -0.74 1.01      351      788
## ndt_condition1      0.26      0.03    0.19    0.33 1.00     1897     2490
##
## Constant Parameters:
##           Value
## s_Intercept    0.00
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Moreover, we test whether the estimates for the drift rate and non-decision time are substantial:

```
ezdm_hypothesis <- c(
  driftFX = "drift_condition1 = 0",
  ndtFX = "ndt_condition1 = 0"
)

ezdm_hypothesis_results <- brms::hypothesis(
  ezdm_fit,
  ezdm_hypothesis
)

plot(ezdm_hypothesis_results)
```



```
1/abs(ezdm_hypothesis_results$hypothesis$Evid.Ratio)
```

```
## [1] 5.581679e+64      Inf
```